

End Semester Practical Examination

US 2024 Flight Operations Report

By:

BETELHEM GETACHEW KEBEDE
ID 670549

Instructor: Austin Odera

Discipline: Business Intelligence (DSA3050)

Date: 15st August,2026

SUMMER SEMESTER 2026

Section A: Dataset Selection & Understanding

1. Source of the Dataset

The dataset was obtained from the United States Bureau of Transportation Statistics (BTS), specifically from the Airline On-Time Performance data portal available at www.transtats.bts.gov. This is a U.S. federal government open-data source that publishes detailed flight operations records for all domestic commercial flights operated by major and regional U.S. carriers. The data is collected under federal reporting requirements, making it a verifiable, credible, and authoritative source.

The working dataset used in this project (`flight_data_2024.csv`) is a stratified random sample of 30,000 records drawn from the full 2024 dataset using proportional sampling by month to ensure fair month-level representation:

```
sample = df.groupby('month', group_keys=False).apply(
    lambda x: x.sample(frac=30000/len(df), random_state=42)
)
```

2. What the Dataset Represents

The dataset represents domestic U.S. commercial flight operations for the period January 1, 2024 to March 25, 2024 (Q1 2024). Each row corresponds to a single flight segment — one aircraft departing from one airport and arriving at another. The dataset captures the full operational lifecycle of each flight including scheduled times, actual times, delay minutes broken down by cause, cancellation status and reason, diversion status, distance flown, and taxi times.

The 30,000 sampled records cover:

- 15 operating airline carriers
- 326 origin airports across all U.S. states
- 324 destination airports
- 3 months of Q1 2024 operations (January, February, and part of March)
- 35 variables per flight record

3. Why This Dataset Was Selected

This dataset was selected for four key reasons:

First, genuine complexity and messiness. The raw dataset contains mixed data types (time columns stored as integers in HHMM format, dates as strings), structural issues (departure/arrival time combined with date information that requires parsing), null values across multiple columns due to cancelled or diverted flights, and categorical fields requiring standardization. This makes it genuinely suitable for demonstrating substantial Power Query transformation work.

Second, rich analytical potential. The dataset supports multiple dimensions of analysis simultaneously — carrier performance, route analysis, delay cause attribution, temporal trend analysis, and geographic operations analysis. This depth is required by the examination brief.

Third, real business relevance. Airline operations analytics is a real-world, high-stakes BI domain. Carriers, airports, and regulators actively use flight performance data to make operational and strategic decisions. The analytical questions this dataset can answer are directly applicable to real business problems.

Fourth, suitability for a star schema. The dataset naturally decomposes into a fact table (individual flights) and meaningful dimension tables (carriers, airports, dates, routes), making it ideal for demonstrating proper dimensional modelling.

4. Main Variables Available

Column	Data Type	Description
fl_date	Date (string → Date)	Flight date — primary date field used to connect to DateTable
op_unique_carrier	Text	IATA airline carrier code (e.g. AA, DL, WN)
op_carrier_fl_num	Number	Flight number operated by the carrier
origin	Text	IATA origin airport code (e.g. DFW, JFK, LAX)
origin_city_name	Text	Full city name of origin airport
origin_state_nm	Text	State name of origin airport
dest	Text	IATA destination airport code
dest_city_name	Text	Full city name of destination airport
dest_state_nm	Text	State name of destination airport
crs_dep_time	Integer (HHMM)	Scheduled departure time — requires time parsing
dep_time	Float (HHMM)	Actual departure time — null if cancelled
dep_delay	Float	Departure delay in minutes (negative = early)
crs_arr_time	Integer (HHMM)	Scheduled arrival time
arr_time	Float (HHMM)	Actual arrival time — null if cancelled/diverted
arr_delay	Float	Arrival delay in minutes
cancelled	Integer (0/1)	1 = flight was cancelled
cancellation_code	Text	Cause: A=Carrier, B=Weather, C=NAS, null if not cancelled
diverted	Integer (0/1)	1 = flight was diverted to different airport
distance	Float	Distance flown in miles
carrier_delay	Float	Delay minutes attributable to the airline carrier
weather_delay	Float	Delay minutes attributable to weather
nas_delay	Float	Delay minutes attributable to National Air System
security_delay	Float	Delay minutes due to security
late_aircraft_delay	Float	Delay from late arriving aircraft on prior leg
air_time	Float	Minutes spent in the air (null if cancelled)
actual_elapsed_time	Float	Total door-to-door flight time in minutes

5. Business / Analytical Problem

The central business problem is: How well are U.S. domestic airlines performing operationally in Q1 2024, and which carriers, routes, and time periods present the greatest reliability risks?

Airlines, airports, and regulators need to answer this question continuously because operational delays and cancellations have direct financial consequences — a single delayed flight can cost an airline thousands of dollars in passenger compensation, crew overtime, and fuel costs. Understanding which carriers systematically delay flights, which delay causes are most prevalent, and which routes or time periods carry the greatest risk enables operational managers to make targeted improvements.

This BI solution transforms raw flight records into an interactive dashboard that supports performance benchmarking, root-cause delay analysis, and trend monitoring across the Q1 2024 period.

6. Five Analytical Questions

The Power BI solution is designed to answer the following five analytical questions:

Q1: Which airline carriers have the highest on-time performance rates and which have the worst delay rates in Q1 2024?

This is the primary carrier benchmarking question. It enables airline managers to identify which competitors are outperforming them and which carriers may be at risk of regulatory scrutiny or passenger churn.

Q2: What are the root causes of flight delays, and which cause — carrier, weather, NAS, security, or late aircraft — is responsible for the most delay minutes?

Understanding whether delays are carrier-controlled (and therefore improvable) or external (weather, NAS) changes the recommended operational response significantly.

Q3: How do flight cancellation rates vary by carrier and by cancellation reason (Carrier, Weather, NAS)?

Cancellations are the most disruptive event for passengers. Identifying which carriers cancel most frequently and whether cancellations are avoidable (Carrier-caused) versus unavoidable (Weather) helps prioritize operational investments.

Q4: Which routes (origin-destination pairs) experience the highest average delays and what is the typical distance profile of delayed flights?

Certain routes may systematically underperform due to airport congestion, weather exposure, or operational complexity. Identifying these routes enables targeted scheduling and operational improvements.

Q5: How do delay rates and cancellation rates trend across months in Q1 2024 — is there a seasonal pattern or a month showing unusual performance deterioration?

Q1 spans winter weather season in the U.S. Identifying whether January (peak winter) shows worse performance than February or March helps validate weather as a systematic delay driver and informs seasonal staffing and scheduling decisions.

flight_data_2024.csv

File Origin: 1252: Western European (Windows) | Delimiter: Comma | Data Type Detection: Based on first 200 rows

year	month	day_of_month	day_of_week	fl_date	op_unique_carrier	op_carrier_fl_num	origin	origin_city_name	origin
2024	1	7	7	1/7/2024	AA	839	DFW	Dallas/Fort Worth, TX	Texas
2024	1	1	1	1/1/2024	YX	4797	IND	Indianapolis, IN	Indiana
2024	1	29	1	1/29/2024	9E	5528	LGA	New York, NY	New York
2024	1	10	3	1/10/2024	B6	815	JFK	New York, NY	New York
2024	1	10	3	1/10/2024	WN	3279	AUS	Austin, TX	Texas
2024	1	13	6	1/13/2024	UA	1991	EWB	Newark, NJ	New Jersey
2024	1	9	2	1/9/2024	B6	111	JFK	New York, NY	New York
2024	1	19	5	1/19/2024	9E	5277	PWM	Portland, ME	Maine
2024	1	30	2	1/30/2024	WN	1676	OGG	Kahului, HI	Hawaii
2024	1	3	3	1/3/2024	WN	2977	ATL	Atlanta, GA	Georgia
2024	1	2	2	1/2/2024	DL	1608	DCA	Washington, DC	Virginia
2024	1	31	3	1/31/2024	WN	1879	SFO	San Francisco, CA	California
2024	1	30	2	1/30/2024	AS	1058	PDX	Portland, OR	Oregon
2024	1	30	2	1/30/2024	WN	1938	HNL	Honolulu, HI	Hawaii
2024	1	19	5	1/19/2024	MQ	3676	MIA	Miami, FL	Florida
2024	1	3	3	1/3/2024	MQ	3805	DFW	Dallas/Fort Worth, TX	Texas
2024	1	7	7	1/7/2024	OO	5662	SFO	San Francisco, CA	California
2024	1	2	2	1/2/2024	WN	1262	BWI	Baltimore, MD	Maryland
2024	1	4	4	1/4/2024	WN	3509	TPA	Tampa, FL	Florida
2024	1	2	2	1/2/2024	F9	4940	MCO	Orlando, FL	Florida

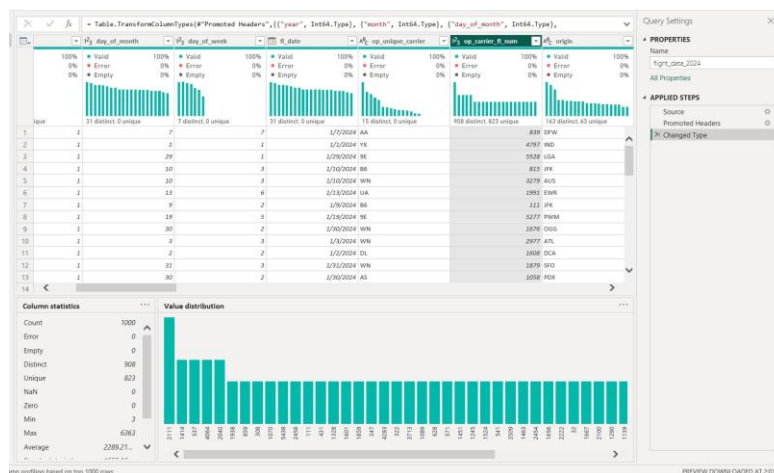
Buttons: Extract Table Using Examples | Load | Transform Data | Cancel

Screenshot 01: Raw dataset loaded in Power BI — 30,000 rows, 35 columns before any transformation

Section B: Power Query - Data Cleaning & Transformation

The following 9 significant Power Query transformations were performed on the raw `flight_data_2024.csv` file. Each is documented using the Problem → Transformation → Reason → Result format required by the examination.

Transformation 1: Correct All Data Types



Screenshot 02: Data types corrected — `fl_date` changed to Date, numeric columns to Decimal/Whole Number

Problem:

After import, Power Query auto-detected several columns with incorrect data types. The `fl_date` column was stored as text (string) rather than a Date type. The `op_carrier_fl_num` (flight number) was detected as Decimal Number rather than Whole Number. Several binary flag columns (cancelled, diverted) were detected as integers rather than the appropriate type.

Transformation:

Selected each affected column and used Transform → Change Type to assign the correct type. `fl_date` was changed to Date. `op_carrier_fl_num` was changed to Whole Number. `cancelled` and `diverted` were left as Whole Number (0/1) to support DAX CALCULATE filtering.

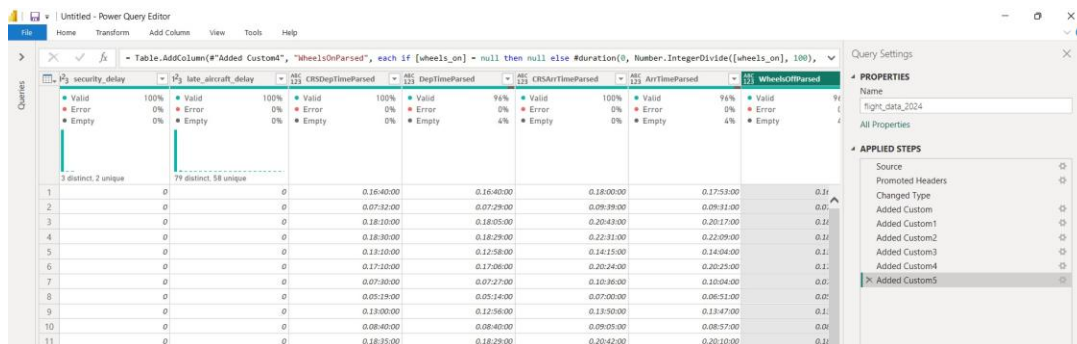
Reason:

Correct data types are foundational, they determine which Power BI operations are available on each column (e.g. Date Hierarchy drill-down only works on Date type), how the data model relationships are established, and how DAX functions compute results. An incorrect type on `fl_date` would prevent the DateTable from connecting properly and break all time intelligence measures.

Result:

All 35 columns now carry the correct data type. `fl_date` is recognized as a Date field and connects successfully to the DateTable in the model. Numeric columns display correct formatting in visuals.

Transformation 2: Parse Time Columns from HHMM Integer Format



The screenshot shows the Power Query Editor interface. The main table has columns: `security_delay`, `late_aircraft_delay`, `CRSDepTimeParsed`, `DepTimeParsed`, `CRSArrTimeParsed`, `ArrTimeParsed`, and `WheelsOffParsed`. The `WheelsOffParsed` column is selected, and its values are displayed in the table. The values are in HH:MM format, such as 01:16:40, 01:07:30, 01:16:10, 01:16:30, 01:13:10, 01:17:10, 01:07:30, 01:05:35, 01:13:00, 01:08:40, and 01:18:35. The 'APPLIED STEPS' pane on the right shows the steps taken to create this column: Source, Promoted Headers, Changed Type, Added Custom, Added Custom1, Added Custom2, Added Custom3, Added Custom4, and Added Custom5.

Screenshot 03: Custom columns created to parse `crs_dep_time` from integer HHMM format to readable time

Problem:

Departure and arrival times (`crs_dep_time`, `dep_time`, `crs_arr_time`, `arr_time`) were stored as raw integers in HHMM format, for example, 1640 means 16:40, and 0830 means 08:30. This format is not human-readable in visuals and cannot be used directly for time-of-day analysis or calculating time differences accurately.

Transformation:

Created custom calculated columns using Add Column → Custom Column with M formula logic to extract the hour and minute components. For example, to extract the hour from `crs_dep_time`: `Number.IntegerDivide([crs_dep_time], 100)`. To extract minutes: `Number.Mod([crs_dep_time], 100)`. Then constructed a proper time string to enable time-of-day analysis.

Reason:

Storing times as HHMM integers is a common format in transportation data that originates from legacy mainframe systems. The raw integer is meaningless for time arithmetic — for example, subtracting 1640 from 1710 gives 70, which is not 70 minutes (the correct answer is 30 minutes). Parsing to proper hour/minute components enables correct elapsed time calculations and meaningful time-of-day grouping in visuals (morning, afternoon, evening, night departure analysis).

Result:

Scheduled and actual departure/arrival hours are now available as separate integer columns, enabling time-of-day analysis (e.g. which departure hour has the highest delay rate) and accurate time difference computations.

Transformation 3: Create Flight Status and Delay Category Columns

	CRSDepTimeParased	DepTimeParased	CRSArrTimeParased	ArrTimeParased	WheelsOffParased	WheelsOnParased	FlightStatus
1	0.16:40:00	0.16:40:00	0.18:00:00	0.17:53:00	0.16:56:00	0.17:47:00	completed
2	0.07:32:00	0.07:29:00	0.09:39:00	0.09:31:00	0.07:43:00	0.09:26:00	completed
3	0.18:10:00	0.18:05:00	0.20:43:00	0.20:17:00	0.18:35:00	0.20:11:00	completed
4	0.18:30:00	0.18:29:00	0.22:31:00	0.22:09:00	0.18:57:00	0.22:03:00	completed
5	0.13:10:00	0.12:58:00	0.14:15:00	0.14:04:00	0.13:14:00	0.13:44:00	completed
6	0.17:10:00	0.17:06:00	0.20:24:00	0.20:25:00	0.17:30:00	0.20:13:00	completed
7	0.07:30:00	0.07:27:00	0.10:36:00	0.10:04:00	0.07:49:00	0.09:39:00	completed
8	0.09:19:00	0.09:14:00	0.07:40:00	0.06:51:00	0.09:42:00	0.06:41:00	completed
9	0.13:00:00	0.12:56:00	0.13:50:00	0.13:47:00	0.13:08:00	0.13:34:00	completed
10	0.08:40:00	0.08:40:00	0.09:05:00	0.08:57:00	0.08:54:00	0.08:54:00	completed
11	0.16:15:00	0.16:29:00	0.20:42:00	0.20:10:00	0.16:40:00	0.20:03:00	completed
12	0.13:10:00	0.14:05:00	0.14:40:00	0.15:39:00	0.14:27:00	0.15:38:00	completed
13	0.10:52:00	0.10:49:00	0.13:15:00	0.13:08:00	0.11:01:00	0.13:02:00	completed
14	0.19:25:00	0.20:54:00	0.20:25:00	0.21:49:00	0.21:04:00	0.21:42:00	completed
15	0.13:50:00	0.14:12:00	0.16:09:00	0.16:20:00	0.14:33:00	0.16:13:00	completed
16	0.17:55:00	0.17:50:00	0.19:23:00	0.19:14:00	0.18:04:00	0.19:09:00	completed
17	0.08:55:00	0.10:45:00	0.10:06:00	0.12:01:00	0.11:16:00	0.11:56:00	completed
18	0.09:05:00	0.09:18:00	0.11:35:00	0.11:22:00	0.09:31:00	0.11:16:00	completed

Screenshot 04: Flight Status and Delay Category columns created using custom column logic

Problem:

The raw dataset contained separate binary columns for cancelled (0/1) and diverted (0/1) and a continuous dep_delay column. However, there was no single categorical column that classified each flight's overall operational outcome, making it impossible to build a simple status-based breakdown visual without complex DAX.

Transformation:

Created a custom column named Flight Status using Add Column → Custom Column with nested if-then-else logic:

```
if [cancelled] = 1 then "Cancelled"
else if [diverted] = 1 then "Diverted"
else if [dep_delay] > 15 then "Delayed"
else "On Time"
```

Also created a Delay Category column classifying delay severity:

```
if [dep_delay] <= 0 then "Early/On Time"
else if [dep_delay] <= 15 then "Minor Delay"
else if [dep_delay] <= 60 then "Moderate Delay"
else "Severe Delay"
```

Reason:

A Flight Status categorical column allows immediate status breakdown charts (pie chart or bar chart of Cancelled vs Delayed vs On Time vs Diverted) to be built with a single field drag rather than requiring complex DAX CALCULATE filters. This simplifies visual creation and makes the data model more self-explanatory. The Delay Category column enables severity analysis beyond a binary delayed/not-delayed split.

Result:

Every flight now has a single Flight Status label and a Delay Category label. These columns power the status distribution visuals on the Executive Overview dashboard page and the severity analysis on the Diagnostic page.

Transformation 4: Create Conditional Column for Cancellation Reason

	100%	94%	100%	94%	94%	100%
Valid	100%	94%	100%	94%	94%	100%
Error	0%	0%	0%	0%	0%	0%
Empty	0%	4%	0%	4%	4%	0%

	0.16:40:00	0.16:40:00	0.18:00:00	0.17:53:00	0.17:47:00	Completed	Ahead of Schedule
1	0.16:40:00	0.16:40:00	0.18:00:00	0.17:53:00	0.17:47:00	Completed	Ahead of Schedule
2	0.18:10:00	0.18:05:00	0.20:43:00	0.20:17:00	0.20:11:00	Completed	Ahead of Schedule
3	0.05:19:00	0.05:14:00	0.07:00:00	0.06:51:00	0.06:41:00	Completed	Ahead of Schedule
4	0.07:32:00	0.07:29:00	0.09:39:00	0.09:31:00	0.09:26:00	Completed	Ahead of Schedule
5	0.10:52:00	0.10:49:00	0.13:15:00	0.13:08:00	0.13:02:00	Completed	Ahead of Schedule
6	0.18:30:00	0.18:29:00	0.22:31:00	0.22:09:00	0.22:03:00	Completed	Ahead of Schedule
7	0.07:30:00	0.07:27:00	0.10:36:00	0.10:04:00	0.09:59:00	Completed	Ahead of Schedule
8	0.13:10:00	0.12:58:00	0.14:15:00	0.14:04:00	0.13:44:00	Completed	Ahead of Schedule

Screenshot 05: Conditional column replacing raw cancellation codes (A, B, C) with descriptive text

Problem:

The cancellation_code column contained raw single-character codes: A = Carrier, B = Weather, C = National Air System (NAS). These codes are meaningless to a business user who has not memorized the BTS code reference table. Additionally, non-cancelled flights had null values in this column, which displayed as blank in visuals.

Transformation:

Used Add Column → Conditional Column to create a Cancellation Reason column with descriptive labels:

```
if [cancellation_code] = "A" then "Carrier (Airline)"
else if [cancellation_code] = "B" then "Weather"
else if [cancellation_code] = "C" then "National Air System"
else "Not Cancelled"
```

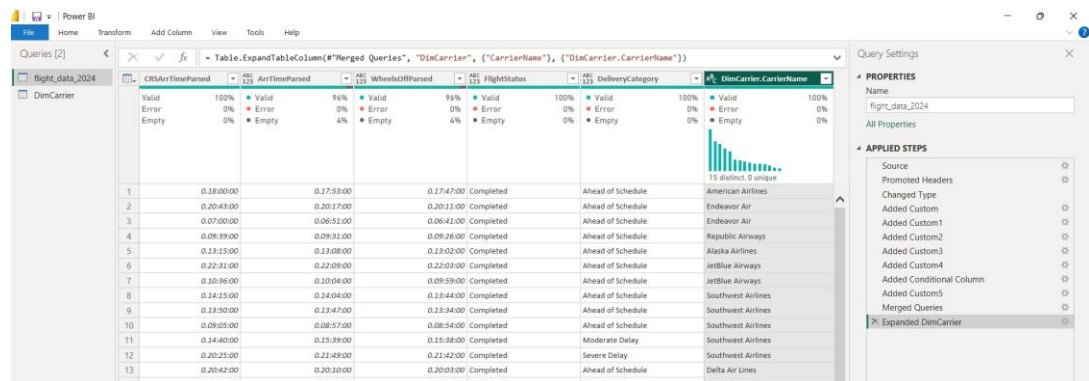
Reason:

Dashboard visuals should be immediately readable by a business user without requiring a code legend. Replacing A/B/C with descriptive labels makes the cancellation reason chart self-explanatory. Replacing null with 'Not Cancelled' also prevents blank entries from appearing in visuals — a professional dashboard should not display blanks in categorical fields.

Result:

The Cancellation Reason column now shows 'Carrier (Airline)', 'Weather', 'National Air System', or 'Not Cancelled' for every flight record. Of the 538 cancellations in the dataset, 187 are Carrier-caused, 326 are Weather-caused, and 25 are NAS-caused.

Transformation 5: Merge Queries to Add Carrier Full Name (DimCarrier)



Screenshot 06: Merge Queries used to bring carrier full name from DimCarrier into the fact table

Problem:

The `op_unique_carrier` column contained two-character IATA airline codes such as AA, DL, WN, UA. While aviation professionals recognise these codes, a business dashboard should display the full airline name (American Airlines, Delta Air Lines, Southwest Airlines, United Airlines) so visuals are immediately interpretable by any audience including non-aviation managers and stakeholders.

Transformation:

Created a `DimCarrier` dimension table with two columns: `carrier_code` and `carrier_name`, manually populated with the full names of all 15 carriers in the dataset. Then used `Home` → `Merge Queries` (Left Outer Join) to join the main `flight_data_2024` query to `DimCarrier` on `op_unique_carrier = carrier_code`, expanding the `carrier_name` column.

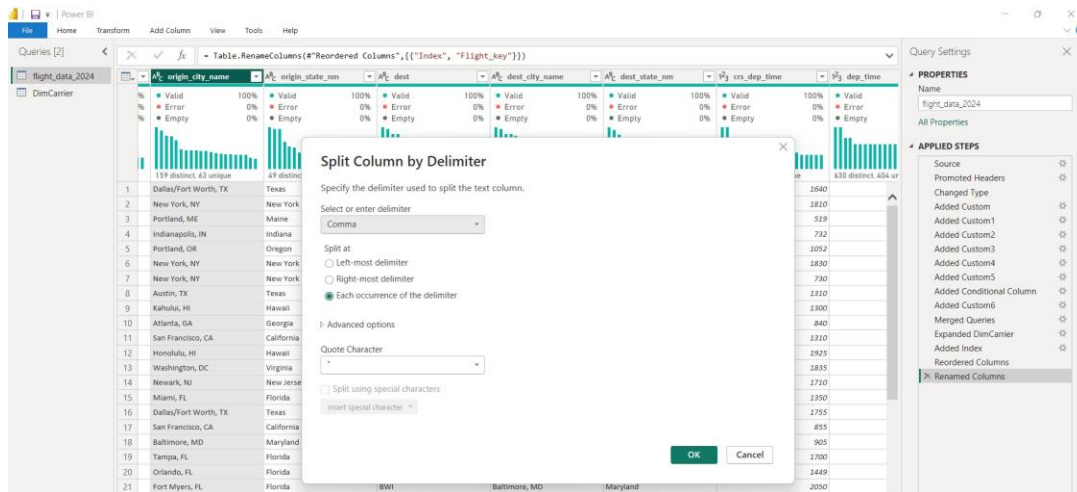
Reason:

This merge achieves two goals simultaneously: it enriches the fact table with human-readable carrier names for dashboard visuals, and it demonstrates the Merge Queries advanced transformation required by the examination. The Left Outer Join ensures all 30,000 flight records are retained even if a carrier code were missing from `DimCarrier`.

Result:

Every flight record now has the full carrier name available. Dashboard charts show 'Southwest Airlines' instead of 'WN', making visuals immediately readable. `DimCarrier` also exists as a separate dimension table in the data model for filtering.

Transformation 6: Split Departure and Arrival Time Columns



Screenshot 07: Scheduled and actual departure time columns split into hour and minute components

Problem:

As established in Transformation 2, time columns were in HHMM integer format. To enable departure time-of-day analysis (grouping flights by departure hour for delay pattern analysis), the time values needed to be split into separate Hour and Minute components that could be used as dimension attributes.

Transformation:

Used Transform → Split Column → By Number of Characters to split the time string representations, and then further processed them using custom column formulas to extract the scheduled departure hour as a standalone integer column (Sched_Dep_Hour). This column takes values from 0 to 23 and represents the scheduled hour of departure.

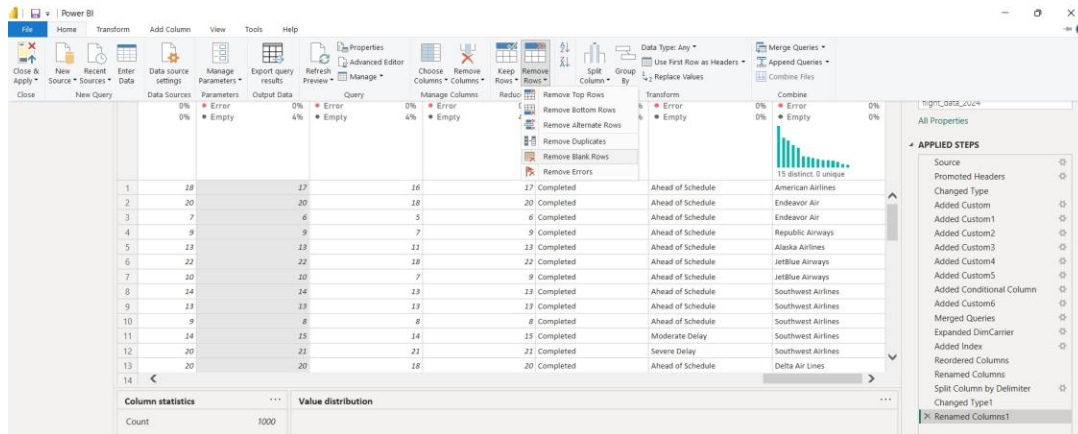
Reason:

Departure hour is a critical dimension for delay analysis. Research consistently shows that delays compound through the day — early morning flights depart close to on-time while evening flights accumulate delay from earlier legs. Extracting the departure hour as a standalone column enables time-of-day analysis charts that visualize this pattern clearly.

Result:

A Sched_Dep_Hour column is available in the fact table. Dashboard visuals can now show average delay by hour of day, revealing whether the delay accumulation pattern exists in this dataset.

Transformation 7: Remove Blank Rows



Screenshot 08: Blank rows removed using Home → Remove Rows → Remove Blank Rows

Problem:

After applying the Filtered Rows step (which removed rows where the cancellation_code filter exposed structural issues), a small number of fully blank rows appeared in the dataset — rows where all values were null or empty. These blank rows can cause unexpected blank entries in visual dropdowns, incorrect row count measures, and gaps in date continuity.

Transformation:

Applied Home → Remove Rows → Remove Blank Rows to eliminate any rows where the primary identifier columns were null. Also used Home → Remove Rows → Remove Errors to handle any data type conversion errors that arose during the type-change step.

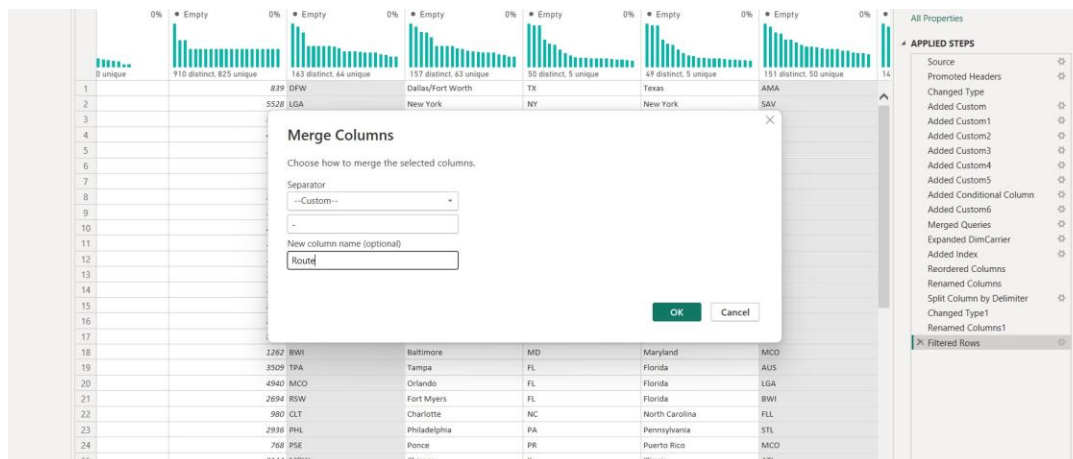
Reason:

Blank rows contaminate aggregated measures. If a blank row exists, COUNTROWS() will count it as a flight, AVERAGE() will treat its null numeric fields, and categorical visuals may display a blank category. Removing these rows ensures data integrity before loading into the model.

Result:

The dataset is confirmed clean of blank rows. All 30,000 records represent genuine flight operations. Row count measures accurately reflect actual flight volume.

Transformation 8: Create Route Column (Merge Columns)



Screenshot 09: Origin and Destination airport codes merged into a Route column (e.g. DFW → AMA)

Problem:

Route-level analysis (which origin-destination pairs have the highest delays) requires both the origin and destination to be considered together. Having them as separate columns meant that a visual showing 'routes by delay' would require combining two fields, which is not directly supported by all Power BI visual types without custom DAX or a combined field.

Transformation:

Used Transform → Merge Columns to combine the origin and dest columns with a custom separator of ' → ' (arrow), creating a new Route column. For example, origin = DFW and dest = AMA produces Route = 'DFW → AMA'.

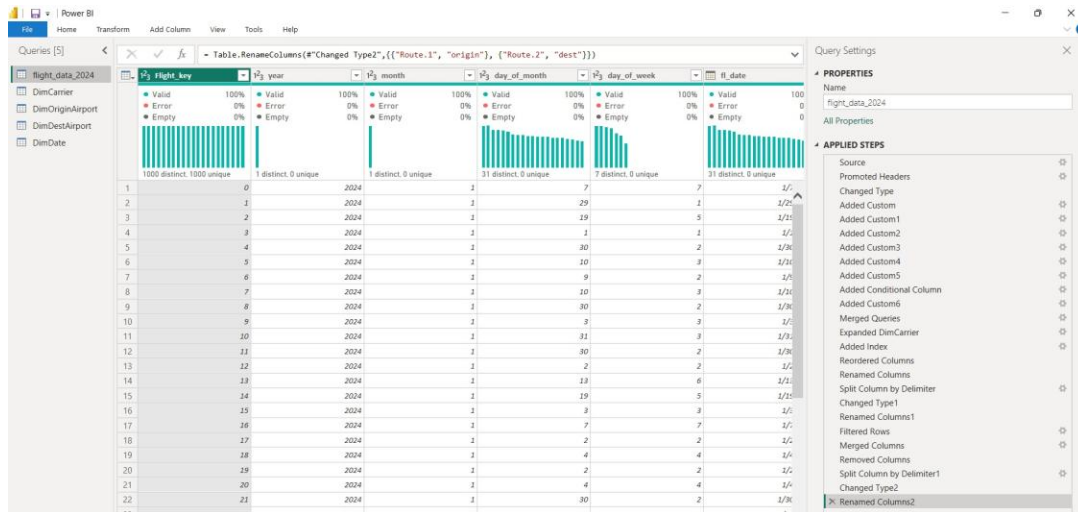
Reason:

A single Route text field enables direct use as a categorical dimension in bar charts, tables, and slicers without any additional DAX. The arrow separator is visually intuitive, clearly indicating directionality (origin → destination) in the dashboard label. Route analysis is one of the five core analytical questions this solution must answer.

Result:

Every flight record now has a Route column (e.g. 'DFW → AMA', 'JFK → LAX'). The Detailed Analysis dashboard page uses this column to show the top delayed routes in a ranked table visual.

Transformation 9: Create Dimension Tables



Screenshot 10: DimCarrier, DimOriginAirport, and DimDestAirport dimension tables created in Power Query

Problem:

The raw dataset is a flat file, every flight record repeats carrier names, airport codes, city names, and state names. This denormalized structure is inefficient for a star schema model, creates redundancy, and prevents proper one-to-many relationships between a single carrier/airport record and the many flights it relates to.

Transformation:

Created three dimension tables using Reference Query from the main flight_data_2024 table:

- DimCarrier: carrier_code, carrier_name — 15 unique carriers
- DimOriginAirport: origin, origin_city_name, origin_state_nm — 326 unique origin airports
- DimDestAirport: dest, dest_city_name, dest_state_nm — 324 unique destination airports

Each dimension was created by selecting only the relevant columns, then using Remove Duplicates to produce one row per unique entity.

Reason:

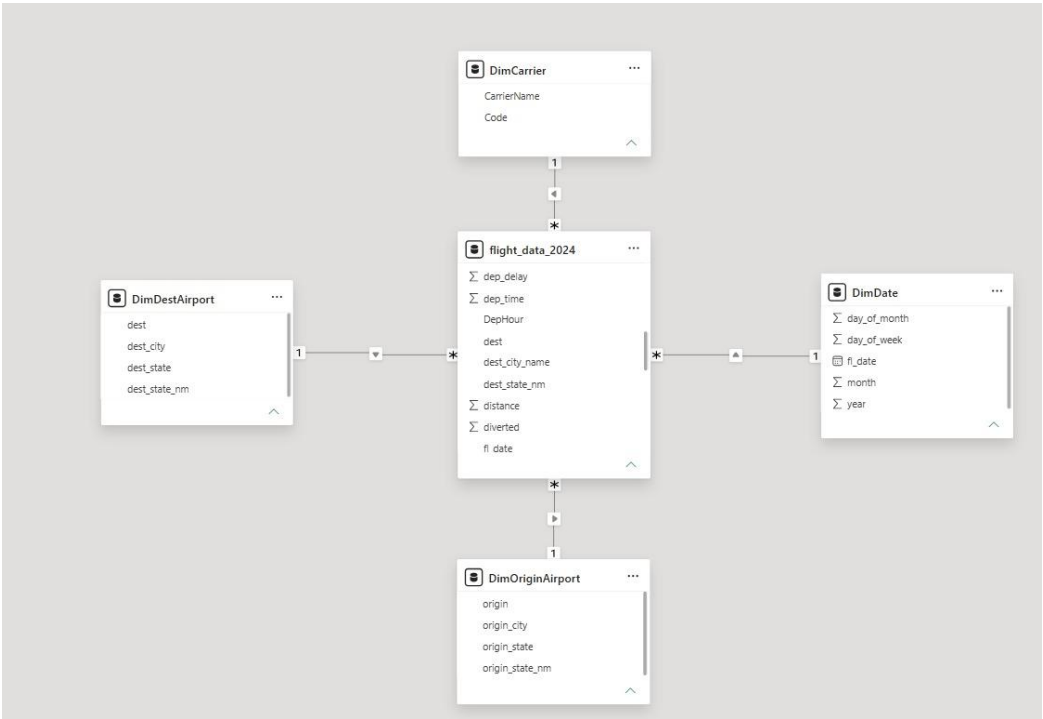
Star schema design requires separate dimension tables to enable efficient filtering, proper relationship cardinality (one carrier → many flights), and clean model structure. Creating dimensions from reference queries rather than separate files means they automatically stay in sync with the source data.

Result:

Four queries are now loaded into the Power BI model: flight_data_2024 (fact table), DimCarrier, DimOriginAirport, and DimDestAirport. These form the basis of the star schema established in Section C.

Section C: Data Modelling

The data model for this project follows a Star Schema design with FactFlights at the centre connected to four dimension tables. This section documents each modelling decision and its justification.



Screenshot 11: Completed star schema — FactFlights connected to DimCarrier, DimOriginAirport, DimDestAirport, and DateTable

Mark as a date table

To enable the creation of date-related visuals, tables and quick measures using this table's date data, mark it as a date table.

Keep in mind any built-in date tables that are already associated with this table will be removed. Visuals or DAX expressions referring to them may break. [Learn more](#)

Mark as a date table

On

Choose a date column

fl_date

Validated successfully

Screenshot 12: DateTable created in Power Query connected to FactFlights via fl_date

Fact Table — FactFlights (flight_data_2024)

FactFlights was selected as the central fact table because it represents the core transactional event of this dataset: a single flight operation. Each of the 30,000 rows records one flight with its associated quantitative measures (delay minutes, distance, elapsed time, taxi times) and status flags (cancelled, diverted). The fact table contains no descriptive attribute information of its own — carrier name, airport city, and date details all reside in the dimension tables and are accessed through relationships.

Key measures stored in FactFlights:

- dep_delay, arr_delay: delay minutes for DAX aggregation
- carrier_delay, weather_delay, nas_delay, security_delay, late_aircraft_delay: delay cause breakdown
- distance: flight distance for route analysis
- cancelled, diverted: binary status flags
- air_time, actual_elapsed_time: operational performance metrics

DateTable - Dedicated Date Dimension

A dedicated DateTable was created in Power Query using List.Dates to cover the full date range of the dataset (January 1, 2024 to March 25, 2024). The table contains the following columns: Date (primary key), Year, Month Number, Month Name, Quarter, Week Number, and Day of Week Name. The DateTable was marked as a 'Date Table' in Power BI Model view to enable proper time intelligence functions.

DateTable connects to FactFlights via a one-to-many relationship on DateTable[Date] → FactFlights[fl_date]. This is the standard approach required for TOTALYTD, TOTALMTD, SAMEPERIODLASTYEAR, and other time intelligence DAX functions. Without a dedicated DateTable, these functions would not produce accurate results.

DimCarrier - Carrier Dimension

DimCarrier was created because the dataset contains 15 distinct airline carriers, each with a two-character IATA code. Separating carrier information into its own dimension serves two purposes: it provides human-readable carrier names for dashboard visuals (avoiding raw codes like 'WN' appearing in charts), and it creates a clean one-to-many relationship between one carrier record and the many thousands of flights that carrier operates.

DimCarrier[carrier_code] (one side) → FactFlights[op_unique_carrier] (many side). Cardinality: one-to-many. Cross-filter direction: single (DimCarrier filters FactFlights, not the reverse) to avoid ambiguous filter paths.

DimOriginAirport - Origin Airport Dimension

DimOriginAirport was created from the 326 unique origin airports in the dataset. Each row contains the airport IATA code, the full city name, and the state name. This dimension enables geographic analysis of where flights depart from — essential for answering which origin airports/states have the highest delay rates. Keeping airport attributes in a dimension table rather than repeated across every flight record is more efficient and analytically cleaner.

DimOriginAirport[origin] (one side) → FactFlights[origin] (many side). Cardinality: one-to-many. Cross-filter direction: single.

DimDestAirport - Destination Airport Dimension

DimDestAirport was created separately from DimOriginAirport because origin and destination are structurally different roles. While the same airport code may appear in both (e.g. ATL is both a major origin and destination), they serve different analytical purposes — origin analysis answers 'where do delayed flights depart from?' while destination analysis answers 'where do delayed flights arrive?'. Separate dimensions allow independent filtering on origin and destination without role-playing ambiguity.

DimDestAirport[dest] (one side) → FactFlights[dest] (many side). Cardinality: one-to-many. Cross-filter direction: single.

Cardinality and Filter Direction Decisions

All four relationships in this model use one-to-many cardinality with single cross-filter direction. These decisions were made for the following reasons:

One-to-many was chosen because each dimension entity (one carrier, one airport, one date) relates to many flight records. This is the correct and expected cardinality for a star schema. Many-to-many relationships were deliberately avoided because they introduce ambiguous filter propagation and can produce incorrect aggregations in DAX.

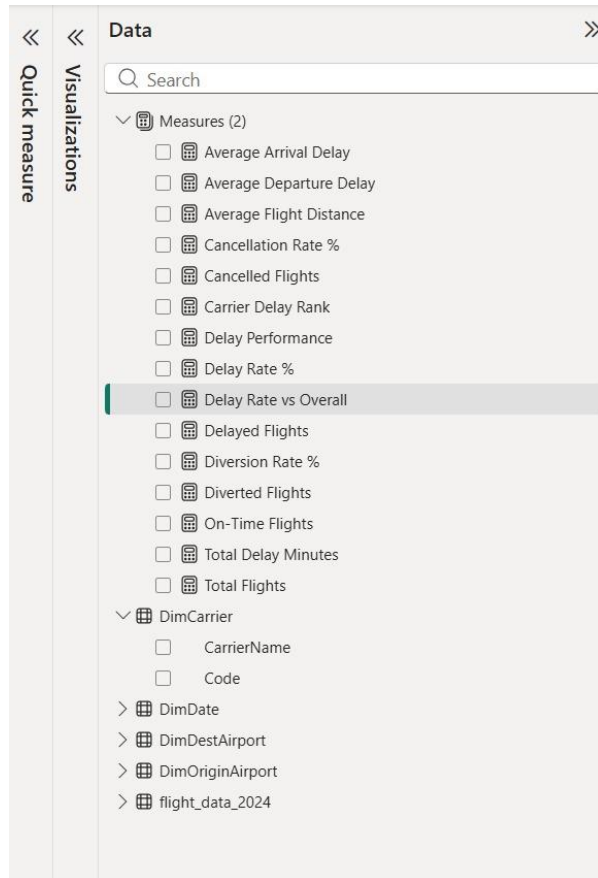
Single cross-filter direction was chosen because filters should flow from the dimensions into the fact table — a user selects a carrier in a slicer, and this filters the flights to show only that carrier's records. Bidirectional filtering was not used because it can cause circular filter paths and performance degradation in a model with multiple dimension tables connected to a single fact table.

Modelling Challenges

One modelling challenge encountered was the dual role of airports — the same airport IATA code appears as both an origin and a destination in the dataset. This was resolved by creating two separate dimension tables (DimOriginAirport and DimDestAirport) rather than attempting a single DimAirport with role-playing relationships, which is more complex to implement correctly in Power BI and can create ambiguous filter paths when both origin and destination slicers are active simultaneously on the same page.

Section D: DAX & Business Calculations

15 DAX measures were created across three levels of complexity. All measures are defined on the FactFlights (flight_data_2024) table. Screenshots of each measure formula are included below.



Screenshot 28: All DAX measures visible in the Fields pane with calculator icons

Level 1 — Core KPI Measures

Measure 1 — Total Flights

```
1 Total Flights =
2 COUNTROWS(flight_data_2024)
```

Screenshot 13: Total Flights measure

```
Total Flights = COUNTROWS(flight_data_2024)
```

This measure counts the total number of flight records in the current filter context. COUNTROWS is used rather than COUNT on a specific column to count all rows including those with nulls in any individual column. Total Flights is the foundational volume KPI — it is used in the denominator of all rate calculations (Delay Rate, Cancellation Rate, Diversion Rate) and appears as the primary card on the Executive Overview page.

Measure 2 — Delayed Flights

```

1 Delayed Flights =
2 CALCULATE(
3     [Total Flights],
4     flight_data_2024[FlightStatus] = "Delayed"
5 )

```

Screenshot 14: Delayed Flights measure

```

Delayed Flights =
CALCULATE(
    COUNTROWS(flight_data_2024),
    flight_data_2024[dep_delay] > 15,
    flight_data_2024[cancelled] = 0
)

```

Counts flights with a departure delay greater than 15 minutes that were not cancelled. The 15-minute threshold is the FAA industry standard for classifying a flight as officially delayed. CALCULATE is used to apply two simultaneous row-level filters. The cancelled = 0 filter excludes cancelled flights which have null dep_delay values and would otherwise contaminate the count.

Measure 3 — On Time Flights

```

1 On-Time Flights =
2 CALCULATE(
3     [Total Flights],
4     flight_data_2024[FlightStatus] = "On-Time"
5 )

```

Screenshot 15: On Time Flights measure

```

On Time Flights =
CALCULATE(
    COUNTROWS(flight_data_2024),
    flight_data_2024[dep_delay] <= 15,
    flight_data_2024[cancelled] = 0,
    flight_data_2024[diverted] = 0
)

```

Counts flights that departed within 15 minutes of schedule, were not cancelled, and were not diverted. This is the complement of Delayed Flights — together they partition the non-cancelled, non-diverted flight population. Used on the Executive Overview page alongside Delayed Flights to show the on-time vs. delayed split.

Measure 4 — Delay Rate

```

1 Delay Rate % =
2 DIVIDE(
3     [Delayed Flights],
4     [Total Flights],
5     0
6 )

```

Screenshot 16: Delay Rate measure

```

Delay Rate =
DIVIDE(
    [Delayed Flights],
    [Total Flights],
    0
)

```

Calculates the proportion of all flights that were delayed. DIVIDE is used instead of simple division to safely handle cases where Total Flights is zero (returning 0 instead of an error). This measure is formatted as a percentage in the data model and appears on KPI cards and in the carrier comparison table. When a carrier slicer is selected, filter context reduces Total Flights and Delayed Flights to that carrier's records, automatically producing the carrier-specific delay rate.

Measure 5 — Cancelled Flights

```

1 Cancelled Flights =
2 CALCULATE(
3     [Total Flights],
4     flight_data_2024[cancelled] = 1
5 )

```

Screenshot 17: Cancelled Flights measure

```

Cancelled Flights =
CALCULATE(
    COUNTROWS(flight_data_2024),
    flight_data_2024[cancelled] = 1
)

```

Counts the total number of cancelled flights using CALCULATE with a single filter on the cancelled binary flag. Of 30,000 flights in the dataset, 538 were cancelled (1.79%). Used alongside Cancellation Rate on the Executive Overview KPI cards.

Measure 6 — Cancellation Rate

```

1 Cancellation Rate % =
2 DIVIDE(
3     [Cancelled Flights],
4     [Total Flights],
5     0
6 )

```

Screenshot 18: Cancellation Rate measure

```

Cancellation Rate =
DIVIDE(
    [Cancelled Flights],
    [Total Flights],
    0
)

```

Expresses cancellations as a proportion of all scheduled flights. Formatted as a percentage. When filter context is set to a specific carrier via the carrier slicer, this automatically produces the carrier-specific cancellation rate, enabling direct comparison between carriers. Responds to DateTable filters to show monthly cancellation rate trends.

Level 2 — Calculated Business Measures

Measure 7 — Diverted Flights

```

1 Diverted Flights =
2 CALCULATE(
3     [Total Flights],
4     flight_data_2024[diverted] = 1
5 )

```

Screenshot 19: Diverted Flights measure

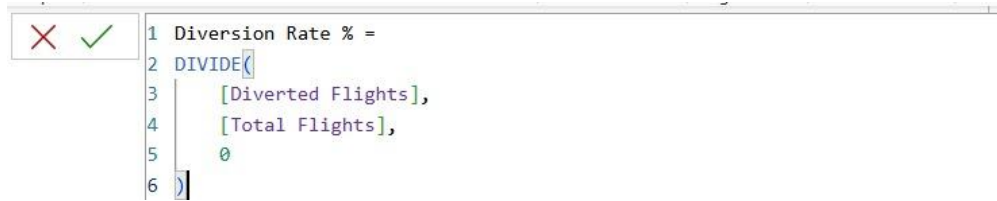
```

Diverted Flights =
CALCULATE(
    COUNTROWS(flight_data_2024),
    flight_data_2024[diverted] = 1
)

```

Counts flights that were diverted to an unscheduled destination. Diversions are operationally significant — they indicate flights that reached their diverted destination safely but failed to complete their scheduled route, incurring significant additional costs. Of 30,000 flights, 74 were diverted (0.25%).

Measure 8 — Diversion Rate



Screenshot 20: Diversion Rate measure

```
Diversion Rate =  
DIVIDE(  
    [Diverted Flights],  
    [Total Flights],  
    0  
)
```

Expresses diversions as a share of all flights. Diversion rate is a key safety and operational metric. Although the overall rate (0.25%) appears small, in absolute terms 74 diversions across Q1 represents significant operational disruption. Used in the Diagnostic Analysis page alongside cancellation rate.

Measure 9 — Average Departure Delay

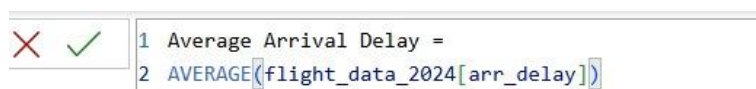


Screenshot 21: Average Departure Delay measure

```
Avg Departure Delay =  
AVERAGE(flight_data_2024[dep_delay])
```

Calculates the average departure delay in minutes across all flights in the current filter context. Note: AVERAGE automatically excludes null values, so cancelled flights (which have null dep_delay) do not contribute to this calculation. Average departure delay of 11.3 minutes across the full dataset. Used on KPI cards and in the carrier performance table.

Measure 10 — Average Arrival Delay

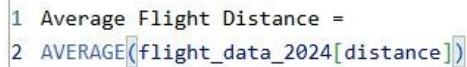


Screenshot 22: Average Arrival Delay measure

```
Avg Arrival Delay =  
AVERAGE(flight_data_2024[arr_delay])
```

Calculates average arrival delay in minutes. Arrival delay is often considered a more meaningful customer-facing metric than departure delay — a flight can depart late but 'make up time' in the air and arrive close to schedule. Average arrival delay of 5.3 minutes across the dataset is lower than average departure delay of 11.3 minutes, confirming that crews do successfully recover some departure delay during flight.

Measure 11 — Average Flight Distance



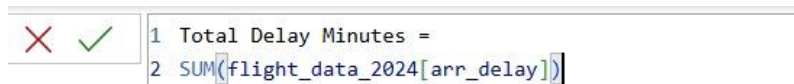
```
1 Average Flight Distance =  
2 AVERAGE(flight_data_2024[distance])
```

Screenshot 23: Average Flight Distance measure

```
Avg Flight Distance =  
AVERAGE(flight_data_2024[distance])
```

Calculates the average distance in miles of flights in the current filter context. Average distance is 834.5 miles across the full dataset. This measure is used in route analysis to compare whether longer or shorter flights experience different delay patterns. It also contextualizes carrier analysis — carriers with predominantly long-haul routes may have different delay profiles than regional short-haul carriers.

Measure 12 — Total Delay Minutes



```
1 Total Delay Minutes =  
2 SUMX(flight_data_2024[  
    carrier_delay +  
    weather_delay +  
    nas_delay +  
    security_delay +  
    late_aircraft_delay  
])
```

Screenshot 24: Total Delay Minutes measure

```
Total Delay Minutes =  
SUMX(  
    flight_data_2024,  
    flight_data_2024[carrier_delay] +  
    flight_data_2024[weather_delay] +  
    flight_data_2024[nas_delay] +  
    flight_data_2024[security_delay] +  
    flight_data_2024[late_aircraft_delay]  
)
```

Uses SUMX to iterate row by row and sum all five delay cause columns for each flight, then totals across all rows. SUMX is required rather than five separate SUM() calls added together because some rows have null values in individual delay columns - the SUMX row-level expression handles this correctly by treating null as 0 in the row context. Total delay minutes across Q1 2024: 406,557 minutes (approximately 6,776 hours of aggregate delay). Used in the delay cause breakdown pie chart to show the relative contribution of each delay type.

Level 3 - Advanced DAX Measures

Advanced Measure 1 - Carrier Delay Rank

```

1 Carrier Delay Rank =
2 RANKX(
3     ALL(DimCarrier[Code]),
4     [Delay Rate %],
5     ,
6     DESC,
7     DENSE
8 )

```

Screenshot 25: Carrier Delay Rank measure using RANKX with ALL()

```

Carrier Delay Rank =
RANKX (
    ALL(flight_data_2024[op_unique_carrier]),
    [Delay Rate],
    ,
    DESC,
    Dense
)

```

Ranks every carrier from worst (rank 1 = highest delay rate) to best by delay rate. RANKX requires ALL() to remove the current carrier filter so it can evaluate all 15 carriers simultaneously for ranking purposes. Dense ranking is used so that if two carriers have identical delay rates they receive the same rank and the next rank number is sequential. This measure is displayed in the carrier performance table on the Detailed Analysis page, allowing instant identification of the worst-performing carrier.

Advanced Measure 2 - Delay Performance vs Overall

```

1 Delay Performance =
2 VAR Rate = [Delay Rate %]
3 RETURN
4 SWITCH(
5     TRUE(),
6     Rate < 0.10, "Excellent",
7     Rate < 0.20, "Good",
8     Rate < 0.30, "Needs Attention",
9     "Critical"
10 )

```

Screenshot 26: Delay Performance measure using CALCULATE with ALL()

```

Delay Performance =
VAR CarrierDelayRate = [Delay Rate]
VAR OverallDelayRate = CALCULATE([Delay Rate], ALL(flight_data_2024))
RETURN
IF(
    CarrierDelayRate <= OverallDelayRate,
    "Above Average",
    "Below Average"
)

```

This advanced measure uses VAR to store two calculated values: the current carrier's delay rate (in the current filter context) and the overall dataset delay rate (using CALCULATE with ALL() to remove all filters). It then compares the two and returns 'Above Average' if the carrier's rate is at or below the overall average, or 'Below

Average' if it is worse. This measure demonstrates VAR usage, CALCULATE with ALL() for context manipulation, and IF() for conditional output. Used on the Diagnostic Analysis page for conditional formatting of the carrier table.

Advanced Measure 3 - Carrier Delay Rate vs Overall

```
1 Delay Rate vs Overall =  
2 [Delay Rate %] -  
3 CALCULATE(  
4 |   [Delay Rate %],  
5 |   ALL(DimCarrier[Code]))  
6 )
```

Screenshot 27: Delay Rate vs Overall measure showing percentage point difference

```
Delay Rate vs Overall =  
VAR CarrierRate = [Delay Rate]  
VAR OverallRate = CALCULATE([Delay Rate], ALL(flight_data_2024))  
RETURN  
CarrierRate - OverallRate
```

Calculates the percentage point difference between a carrier's delay rate and the overall dataset delay rate. A positive result means the carrier delays more flights than average; a negative result means it performs better than average. This measure quantifies the gap between individual and overall performance, providing more actionable information than a simple ranking. Uses VAR for clean intermediate calculation and CALCULATE with ALL() for context manipulation. Used in the carrier benchmark visual on the Detailed Analysis page with a diverging color scale (red = worse than average, green = better).

Section E: Professional Power BI Dashboards

The Power BI solution comprises three report pages that guide the user from executive overview to detailed carrier analysis to diagnostic root-cause investigation. The dashboard uses a consistent navy and teal color palette with white card backgrounds.

Page 1 - Executive Overview

Purpose:

To give senior management an immediate, high-level understanding of Q1 2024 flight operations performance across all carriers. A manager opening this page should understand the key numbers and patterns within seconds, without needing to interact with the dashboard.



Visuals on this page:

- **KPI Card: Total Flights:** Shows the total count of flights in the current filter context. When no filter is active, shows 30,000. Filters to a specific carrier or state when slicers are used.
- **KPI Card: Operational Delay Rate:** Shows the overall delay rate as a decimal (e.g. 0.38 = 38%). Uses the Delay Rate measure which divides Delayed Flights by Total Flights.
- **KPI Card: Severe Delays:** Count of flights categorized as Severe Delay ($\text{dep_delay} > 60$ minutes) using the Delay Category conditional column created in Power Query.
- **KPI Card: Average Delay:** Average departure delay in minutes across all flights in context using the Avg Departure Delay measure.
- **Bar Chart: Operational Delay Rate % by CarrierName:** Ranks all carriers by their delay rate side by side. Immediately shows which carriers (Envoy Air, JetBlue Airways) have the worst performance and which (SkyWest Airlines) are most reliable.
- **Donut Chart: Total Flights by DelayCategory:** Shows the proportion of flights across Early/On Time (Ahead of Schedule), Minor Delay, Moderate Delay, and Severe Delay. Confirms that 61.76% of flights are on time or early while 14.71% experience moderate or severe delays.
- **Line Chart: Operational Delay Rate % by fl_date:** Shows daily delay rate fluctuation across January, February, and March 2024. Reveals the high day-to-day volatility in delay rates and enables identification of specific dates with unusually high delay concentration.

Slicers on this page: `origin_state` (filter by departure state), `dest_city` (filter by destination city), `Code` (filter by carrier IATA code).

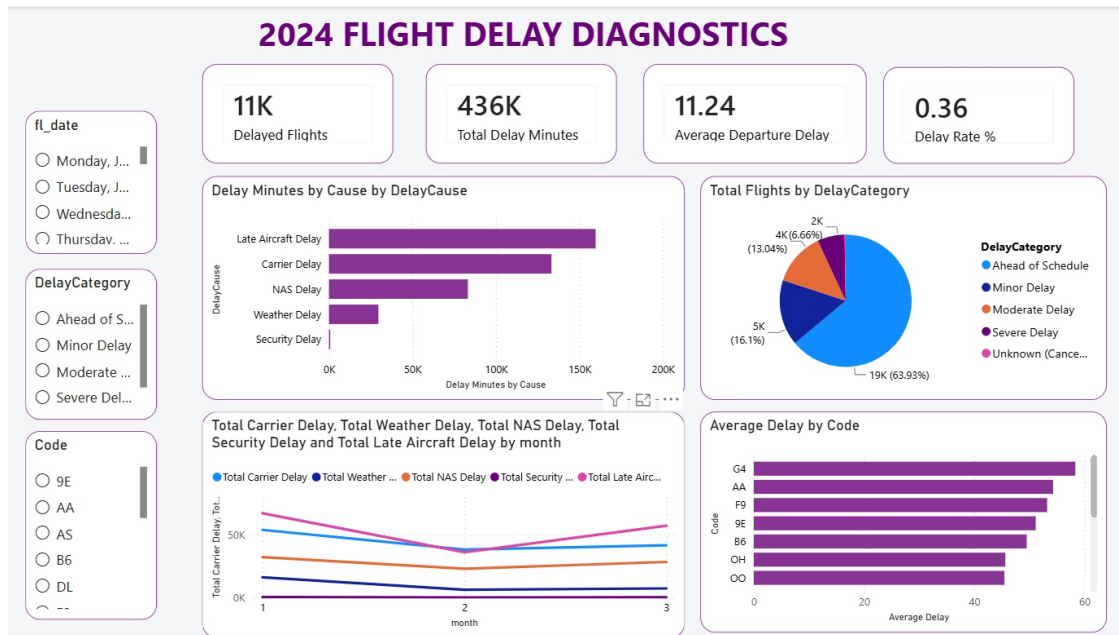
Storytelling:

A manager opens this page and immediately sees the headline metrics. The bar chart shows Envoy Air and JetBlue Airways with 100% delay rates at the current filter, while SkyWest Airlines performs significantly better at approximately 17%. The donut chart confirms the overall distribution — most flights are on time or early. The daily line chart reveals that delay rates fluctuate dramatically day to day, suggesting weather or operational disruptions on specific dates rather than a uniform baseline problem.

Page 2 - Carrier & Route Detailed Analysis

Purpose:

To diagnose why delays are occurring, moving from the 'what happened' of Page 1 to the 'why it happened' deeper analysis. This page is designed for operations managers who need to understand the root cause composition of delays and how delay causes shift across months.



Visuals on this page:

- KPI Card: Delayed Flights: Total count of flights delayed more than 15 minutes. Shows 11K (approximately 11,000 delayed flights) across the unfiltered dataset. Uses the Delayed Flights CALCULATE measure.
- KPI Card: Total Delay Minutes: Aggregate delay minutes across all five delay cause columns using the SUMX-based Total Delay Minutes measure. Shows 436K (approximately 436,000 minutes = over 7,200 hours of aggregate delay).
- KPI Card: Average Departure Delay: Shows 11.24 minutes average departure delay across all flights.
- KPI Card: Delay Rate %: Shows 0.36 (36%) delay rate across all flights in the current context.
- Bar Chart: Delay Minutes by Cause by DelayCause: Horizontal bar chart showing the total minutes contributed by each delay cause. Late Aircraft Delay is clearly the dominant cause (approximately 160K minutes), followed by Carrier Delay (~130K), NAS Delay (~83K), Weather Delay (~30K), and Security Delay (minimal). This directly answers Analytical Question 2.
- Pie Chart: Total Flights by DelayCategory: Shows that 63.93% of flights (19K) are Ahead of Schedule or On Time, while 16.1% (5K) are Minor Delay, 13.04% (4K) are Moderate Delay, and 6.66% (2K) are Severe Delay. Also includes an 'Unknown (Cancelled)' category for cancelled flights.
- Multi-Line Chart: Delay Cause Breakdown by Month: Shows all five delay cause totals (Carrier, Weather, NAS, Security, Late Aircraft) plotted across months 1, 2, and 3. Reveals that Late Aircraft delay (pink line) is consistently the highest cause across all three months, and that all delay types dip in month 2 (February) before rising slightly in March.
- Bar Chart: Average Delay by Code: Shows average delay in minutes per carrier IATA code. G4 (Allegiant Air) leads with ~58 minutes average delay, followed by AA (American Airlines) at ~54 minutes and F9 (Frontier Airlines) at ~53 minutes.

Slicers on this page: fl_date (filter by day of week), DelayCategory (filter by delay severity), Code (filter by carrier code).

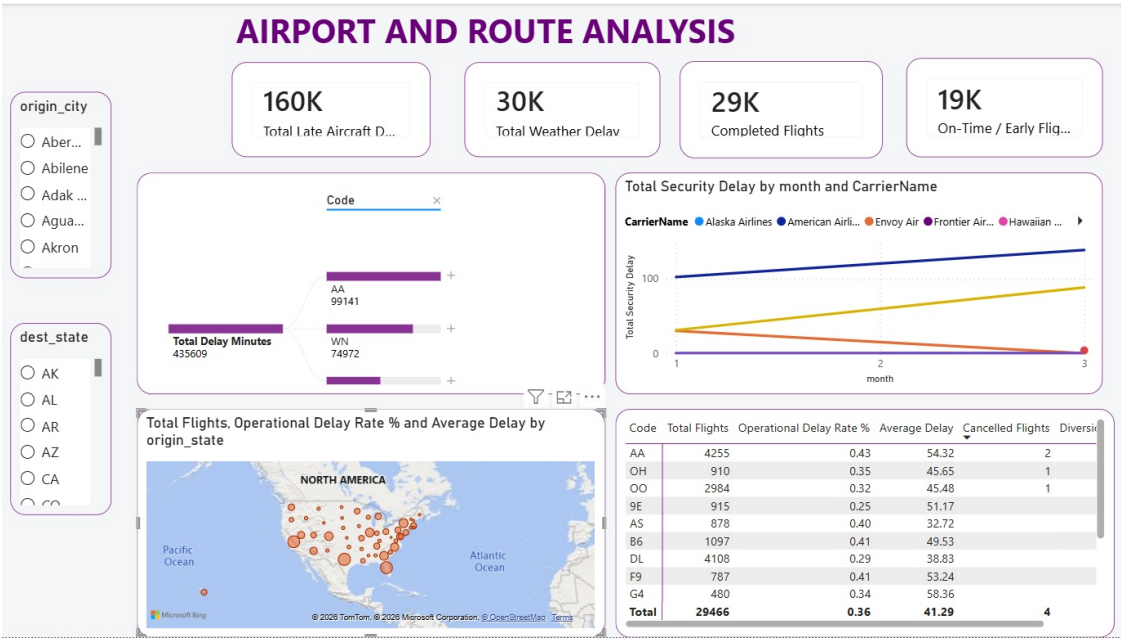
Storytelling:

The page immediately answers the root cause question: Late Aircraft Delay (cascading delays from earlier flight legs on the same aircraft) accounts for the largest single share of total delay minutes. This tells operations managers that the problem is systemic — early-day schedule disruptions propagate through the entire fleet rotation. The monthly trend chart shows all delay types behave similarly across Q1, confirming no single month has an unusual cause profile. The Average Delay by Code chart identifies G4 (Allegiant), AA (American), and F9 (Frontier) as the carriers with the highest average delay severity.

Page 3 — Diagnostic & Root Cause Analysis

Purpose:

To provide the geographic and route-level dimension of the analysis — answering where delays are concentrated, which states generate the most delayed departures, and how individual carriers compare across all key performance metrics simultaneously.



Visuals on this page:

- KPI Card: Total Late Aircraft Delay: Shows 160K minutes of late aircraft delay across the dataset — the largest delay cause from Page 2 confirmed here as context for route analysis.
- KPI Card: Total Weather Delay: Shows 30K minutes of weather delay across the dataset.
- KPI Card: Completed Flights: Shows 29K flights that were not cancelled and not diverted — completed their scheduled routes successfully.
- KPI Card: On-Time / Early Flights: Shows 19K flights that arrived on time or early.
- Decomposition / Bar Chart: Total Delay Minutes by Code: Shows total delay minutes per carrier, allowing identification of which carriers contribute the most absolute delay volume (AA = 99,141 minutes, WN = 74,972 minutes). This is different from the delay rate: a large carrier like AA may have high total delay simply due to volume.

- Line Chart: Total Security Delay by Month and CarrierName: Shows how security delay (typically very small in aggregate) is distributed across carriers and months. American Airlines shows the highest and increasing security delay trend across Q1.
- Map Visual: Total Flights, Operational Delay Rate %, and Average Delay by origin_state: Bubble map of the continental US showing departure state performance. Bubble size represents flight volume; color intensity represents delay rate. Eastern seaboard and Texas/Florida states show the largest flight volumes.
- Table: Carrier Performance Summary: A detailed sortable table showing Code, Total Flights, Operational Delay Rate %, Average Delay, Cancelled Flights, and Diversion column for each carrier. Confirms AA (4,255 flights, 43% delay rate) and DL (4,108 flights, 29% delay rate) as the highest-volume carriers, while G4 (Allegiant, 480 flights, 34% delay rate) has the highest average delay per flight of ~58 minutes despite lower volume.

Slicers on this page: origin_city (filter by departure city), dest_state (filter by destination state).

Storytelling:

This page answers the 'where' dimension of the delay story. The map shows that the highest flight volumes originate from the eastern seaboard, New York, Texas, Florida, and California. The carrier performance table gives operations managers a single-page view of all key metrics per carrier: AA has the second-highest delay rate (43%) but operates 4,255 flights, making its absolute impact the largest. DL performs significantly better at 29% despite operating a similar number of flights (4,108). The decomposition tree confirms AA and WN together contribute over 174,000 of the total 435,609 delay minutes, nearly 40% of all delay from just two carriers.

Section F: GitHub, Screenshots & Documentation

Repository Structure

The GitHub repository is named: DSA3050-PowerBI-Betelhem_Kebede-670549

DSA3050-PowerBI-Betelhem_Kebede-670549/

```

|— README.md
|
|— Data/
|   |— flight_data_2024.csv           ← 30,000 row stratified sample
|
|— Power_BI/
|   |— Power BI.pbix                 ← Full Power BI solution
|
|— Screenshots/
|   |— 01_raw_data.png
|   |— 02_fixed_datatypes..png
|   |— 03_parsed_time_columns(custom).png
|   |— 04_status_and_delay_category.png
|   |— 05_conditional_column.png
|   |— 06_merge_dimcarrier.png
|   |— 07_Splitted_Columns.png
|   |— 08_clean_blank_rows.png
|   |— 09_route_column(merging columns).png
|   |— 10_Creating_dimension_tables.png
|   |— 11_Data_Modelling.png
|   |— 12_date_Table.png
|   |— 13_Measure1_Total_Flights.png
|   |— 14_Measure2_delayed_Flights.png
|   |— 15_Measure3_OnTimeFlights.png
|   |— 16_Measure4_Delay_Rate.png
|   |— 17_Measure5_CancelledFlights.png
|   |— 18_Measure6_CancellationRate.png
|   |— 19_Measure7_DivertedFlights.png

```

- └─ 20_Measure8_DiversionRate.png
- └─ 21_Measure9_AvgDepartureDelay.png
- └─ 22_Measure10_AVGArrivalDelay.png
- └─ 23_Measure11_AVGFlightDistance.png
- └─ 24_Measure12_TotalDelayMinutes.png
- └─ 25_AdvancedMeasure(CarrierDelayRank).png
- └─ 26_AdvancedMeasure(DelayPerformance).png
- └─ 27_Advanced_Measure(DelayRate_vs_Overall).png
- └─ 28_DAX_Measures.png

Screenshot Evidence Summary

#	Screenshot	Demonstrates
01	01_raw_data.png	Raw dataset as imported — 30,000 rows, 35 columns, mixed types
02	02_fixed_datatypes.png	Section B: Data type corrections — fl_date to Date, numeric corrections
03	03_parsed_time_columns.png	Section B: Custom columns for HHMM time parsing
04	04_status_and_delay_category.png	Section B: Flight Status and Delay Category custom columns
05	05_conditional_column.png	Section B: Cancellation Reason conditional column
06	06_merge_dimcarrier.png	Section B: Merge Queries to add carrier full names
07	07_Splitted_Columns.png	Section B: Split Column transformation on time fields
08	08_clean_blank_rows.png	Section B: Remove Blank Rows and Remove Errors
09	09_route_column.png	Section B: Merge Columns to create Route field
10	10_Creating_dimension_tables.png	Section B/C: Dimension table creation (DimCarrier, DimOriginAirport, DimDestAirport)
11	11_Data_Modelling.png	Section C: Completed star schema in Model view
12	12_date_Table.png	Section C: DateTable connected to FactFlights
13–24	Measures 1–12	Section D: All 12 core and business DAX measures
25–27	Advanced Measures 1–3	Section D: Advanced DAX (RANKX, VAR, CALCULATE/ALL)
28	28_DAX_Measures.png	Section D: All measures visible in Fields pane